

Programmazione e calcolo scientifico

Progetto finale

Benedetta Brancato -s325103

Luca Lovisolo - s324445

Marco Ruzzante - s325618

1. Struttura del progetto

Nello sviluppo del progetto abbiamo scelto una struttura modulare che divida nettamente le strutture dati astratte dalla fisica del circuito. Questa scelta riflette la necessità di *disaccoppiare* la struttura del grafo puramente matematica dalle leggi di Kirchhoff e Ohm.

File .hpp (dichiarazioni)

I moduli come graph.hpp e algorithms.hpp sono formati da classi e funzioni template: grafo visto come un'entità matematica pura, che gestisce solo nodi e archi.

File .cpp (definizioni)

I file parser.cpp e circuit_solver.cpp implementano il caso reale richiesto dal progetto. Qui le strutture dati astratte vengono riempite con i dati fisici estratti dalla netlist.

Gestione della memoria:

- **No alla gestione manuale:** Esclusi comandi *new* e *delete*
- **Memoria automatica:** il ciclo di vita di tutti i dati è gestito solo dai contenitori della STL
- **Vantaggi concreti:** Eliminiamo rischio di crash e di spreco di memoria, mantenendo il codice pulito

2. Scelta delle strutture dati

`std::map` (Lista di Adiacenza):

Sostituisce la matrice di adiacenza (troppo densa e inefficiente).

La mappa associativa invece permette di gestire efficientemente i nodi e gli archi. L'uso di un `std::set` per i vicini garantisce un corretto ordinamento dei nodi durante l'esplorazione.

`std::map<double>` (Pesi):

- Struttura **chiave-valore**
- Fornisce accesso immediato ai pesi, operazione fondamentale per l'algoritmo di Dijkstra e il calcolo dei cammini minimi
- Codice pulito ed efficiente

L'invariante di ordinamento dell'arco

Per usare l'arco come chiave nelle mappe, abbiamo optato per un grafo non orientato. Nel file `graph.hpp` forziamo l'invariante geometrico fin dall'inizio

Questo impedisce che l'arco (1,2) e l'arco (2,1) vengano visti come entità separate, eliminando i duplicati

```
undirected_edge(const T& n1, const T& n2) {  
    if (n1 < n2) {  
        nodo_from = n1;  
        nodo_to = n2;  
    } else {  
        nodo_from = n2;  
        nodo_to = n1;  
    }  
}
```

3. Parser e controllo delle anomalie

Il file **parser.cpp** implementa la funzione *leggi_netlist*. L'accoppiamento tra `std::ifstream` con `std::istringstream`: permette di isolare ed analizzare il file del circuito riga per riga. Questa architettura permette di gestire correttamente gli spazi vuoti e, nel caso di una sintassi malformata, garantisce l'arresto controllato del programma. Si evita così di eseguire la simulazione su un circuito errato.

```
while (std::getline(file, riga)) {
    std::istringstream stream(riga);
    Component nuovo_comp;

    if (stream >> nuovo_comp.id >> nuovo_comp.value >> nuovo_comp.node_from >> nuovo_comp.node_to) {
        componenti.push_back(nuovo_comp);
    }

    else {
        bool riga_vuota = true;

        for (char c : riga) {
            if (c != ' ' && c != '\t' && c != '\n') {
                riga_vuota = false;
                break;
            }
        }
    }
}
```

La prevenzione di eventuali problemi del sistema, dopo il parsing, avviene in **main.cpp**:

Controllo Valori Fisici: Se un resistore ha valore uguale a zero, il programma solleva un errore e si arresta immediatamente: una resistenza nulla distruggerebbe la diagonalità positiva della matrice R

Rilevamento Multigrafi: Vengono confrontate le coppie di nodi di ogni nuovo componente, non ammettendo collegamenti in parallelo: in tal caso l'operazione viene bloccata.

4. La Gestione dei Segni e l'Orientamento delle Maglie

A livello informatico il grafo è non orientato per evitare duplicati e ordina gli archi dal nodo **minore** a quello **maggiore**. Tuttavia questa convenzione influenza la fisica del circuito: la corrente è considerata positiva se si muove dal nodo più piccolo a quello più grande

Gli algoritmi di ricerca (DFS o De Pina), esplorano la rete secondo un ordine di visita **non noto a priori**. Per non perdere l'informazione sul verso nella matrice di incidenza B, abbiamo introdotto la struttura **OrientedEdge**: se il cammino segue l'orientamento intrinseco del grafo poniamo $\text{sign} = 1$, se va al contrario poniamo $\text{sign} = -1$.

```
struct OrientedEdge {  
    size_t edge_id;  
    int sign;  
};  
  
using OrientedCycle = std::vector<OrientedEdge>;
```

I generatori ideali vengono mappati nel vettore v **incrociando la polarità** che si ottiene dal parser (node_from positivo, node_to negativo) con i nodi effettivi della maglia.

5. Algoritmo di De Pina: Lifting e Gestione dei Pesi Nulli

La difficoltà principale nello sviluppo del codice ha riguardato il **lifting**, ossia lo sdoppiamento del grafo nei due strati paralleli (+) e (-). Per non modificare la classe del grafo originale, si è introdotta la struttura **VirtualNode**. Poiché in C++ i contenitori `std::set` e `std::map` utilizzano l'operatore `<` per identificare le **chiavi**, se questo operatore si limitasse a confrontare gli ID dei nodi ignorando il segno dello strato, il nodo positivo e quello negativo verrebbero considerati come la stessa entità.

I generatori ideali hanno resistenza nulla, creando nel grafo archi a **peso zero**. In presenza di archi a costo zero appartenenti ai ponti inter-foglio, esistono infiniti cammini a costo totale identico che oscillano tra lo strato (+) e lo strato (-). Introducendo un peso fittizio di 10^{-6} , ogni transizione extra genera un costo addizionale strettamente positivo, evitando loop inutili.

```
template <typename T>
struct VirtualNode {
    T id;
    char sheet_sign;

    bool operator<(const VirtualNode& other) const {
        if (id != other.id) return id < other.id;
        return sheet_sign < other.sheet_sign;
    }
    bool operator>(const VirtualNode& other) const {
        return other < *this;
    }
    bool operator==(const VirtualNode& other) const {
        return id == other.id && sheet_sign == other.sheet_sign;
    }
    bool operator!=(const VirtualNode& other) const {
        return !(*this == other);
    }
};
```

6. Cicli Fondamentali DFS

Un'altra opzione per l'individuazione delle maglie si basa sulla ricerca in profondità (DFS) tramite la funzione `cicli_fondamentali_dfs` in `cycle_basis.hpp`. L'algoritmo estrae in modo ricorsivo uno **Spanning Tree** (albero di copertura); dal calcolo del coalbero, gli **archi esclusi** vengono reinseriti uno alla volta per chiudere un **ciclo fondamentale**. Questo procedimento garantisce l'indipendenza lineare delle maglie trovate, in quanto ogni ciclo contiene una corda che nessun altro ciclo possiede

Problema: la DFS esplora il grafo in profondità seguendo l'ordine dei vicini, e questo genera spesso alberi di copertura molto "sbilanciati" e lunghi. Di conseguenza, i cicli risultanti possono essere molto lunghi e sovrapposti (nessuna garanzia che siano minimi)

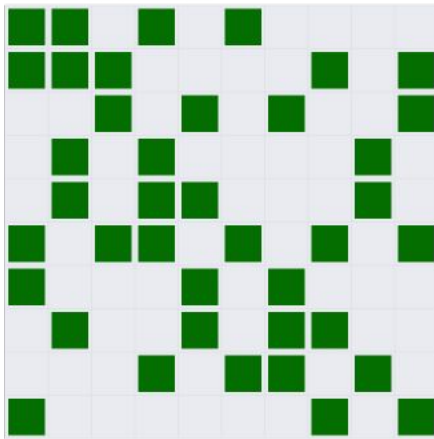
7. Estrazione della base dei cicli

Nella scelta tra DFS e De Pina, occorre ricordare la loro differenza in termini di costo computazionale:

- DFS: È un metodo più veloce ma può produrre maglie molto grandi e sovrapposte, senza garanzia che siano minime. La matrice finale risulta mal condizionata
- De Pina: Estrae una base di cicli minimi. Effettua molte iterazioni, migliorando la condizionatura della matrice finale.

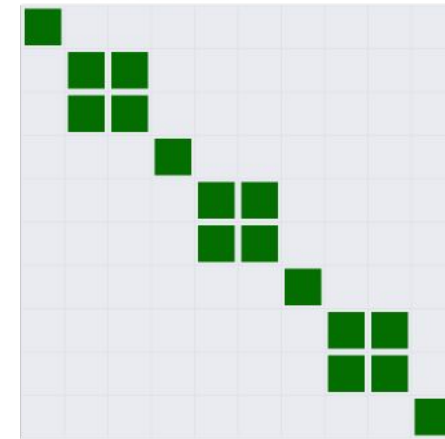
In altre parole, dfs ha un costo computazionale minore ma porta a un sistema lineare molto lungo da risolvere (la matrice A risulta densa). Invece, De Pina ha un costo temporale elevato, ma conduce a un sistema ben condizionato facile da risolvere

Matrice del Sistema A



DFS

Matrice del Sistema A



De Pina

8. Costruzione del Sistema Lineare

Una volta ottenuta la base dei cicli orientati, il `circuit_solver.cpp` avvia la fase di assemblaggio del sistema lineare, che è composto nel seguente modo:

Matrice / Vettore	Dimensione	Descrizione
Resistenze R	MxM	Matrice diagonale contenente i valori in Ohm dei resistori del circuito
Incidenza B	MxN	Mappa le resistenze nelle maglie. Assume valore +1 se il resistore è concorde al verso del ciclo, -1 se discorde, 0 se esterno
Termini Noti v	Nx1	Vettore delle tensioni dei generatori di maglia

dove N è il numero di maglie

$$B = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \dots & b_{mn} \end{bmatrix}$$

$$R = \begin{bmatrix} R_1 & 0 & \dots & 0 \\ 0 & R_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & R_m \end{bmatrix}$$

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

La matrice A del sistema lineare, ottenuta dal calcolo matriciale $A=B'R*B$, presenta le seguenti proprietà

- è quadrata e simmetrica
- è definita positiva

9. Gradiente Coniugato e convergenza

Per la risoluzione del sistema lineare abbiamo implementato il metodo del Gradiente Coniugato nel file header `gradiente_coniugato.hpp`. Trattandosi di un algoritmo iterativo, la sua affidabilità dipende dalle ipotesi sulla matrice A , che deve essere quadrata, simmetrica e definita positiva). Abbiamo quindi strutturato il codice inserendo controlli sulla corretta forma della matrice

```
Eigen::VectorXd Ap = A * p;
double p_Ap = p.dot(Ap);
if (p_Ap < 1e-12) {
    std::cerr << "Errore: La matrice non è definita positiva." << std::endl;
    return std::nullopt;
}
```

L'uso di `std::optional` permette al solutore di comunicare lo stato di fallimento numerico al ``main.cpp`` in modo pulito ed esplicito, senza ricorrere a eccezioni

In caso di convergenza (impostata a una tolleranza di 10^{-10}), le correnti di maglia calcolate vengono moltiplicate per la matrice di incidenza e delle resistenze: `tensioni_resistenze = R * B * correnti_maglie` (Legge di Ohm generalizzata). Questo passaggio finale estrae le cadute di potenziale sui singoli componenti

10. Costi computazionali

Algoritmo	Costo Computazionale	Ruolo
Dijkstra	$O(E \log V)$	Esplorazione del grafo
De Pina	$O(n * (V * E \log V))$	Calcolo dei cicli minimi
Prodotto matrice ($A = B' * R * B$)	$O(n^2 * m)$	Costruzione del sistema lineare
Gradiente Coniugato	$O(\text{iter} * n^2)$	Risoluzione del sistema lineare

Dove **n** è il numero di cicli indipendenti ($n=|E|-|V|+1$) e **m** è il numero dei resistori